

AlgoDSA Platform — System Architecture & Product Blueprint

World-Class AI Learning & FAANG Interview Simulator • System Documentation

1. Executive Summary & Product Vision

AlgoDSA is NOT another LeetCode clone or simple Online Judge. Traditional platforms act strictly as code evaluators (submit code → pass/fail test cases). In contrast, AlgoDSA is an AI-powered learning platform where users learn, think, communicate, and then code.

The core experience revolves around **Alex (Google Senior Staff AI Mentor)**. In real tech interviews at Google, Meta, or Amazon, coding represents only ~30% of the evaluation. The remaining 70% tests communication, edge-case identification, pattern recognition, and trade-off analysis.

Feature Component	Traditional Online Judges (LeetCode/GFG)	AlgoDSA Platform
Core Philosophy	Passive Code Evaluation & Output Comparison	Interactive AI FAANG Interview Simulator
AI Mentor Integration	None or static hint text	Alex — Senior Staff Engineer AI Partner
Interview Workflow	Direct Code Editor access only	Structured 9-Phase Guided Interview Stepper
Multi-Platform Data	Isolated to single platform	Unified Aggregator (AlgoDSA + LC + GFG)
Retention Tracking	Simple count of solved questions	SuperMemo SM-2 Spaced Repetition Engine

2. The 9-Phase Google Interview Stepper

- **Phase 1: Requirements & Constraints:** Clarify problem boundaries, value ranges (e.g. $N \leq 10^5$), and memory constraints.
- **Phase 2: Edge Case Analysis:** Identify edge cases (empty inputs, single elements, negative values, duplicates).
- **Phase 3: Pattern Identification:** Select optimal algorithm pattern (Two Pointers, Sliding Window, DP, Trees, Graphs).
- **Phase 4: Approach & Trade-off Analysis:** Discuss Brute Force $O(N^2)$ vs Optimal $O(N)$ space/time trade-offs.
- **Phase 5: Communicate Algorithm:** Explain plain-English algorithm to Alex (AI Mentor) before coding.
- **Phase 6: Write Code in Monaco Hero Editor:** Write production-grade source code with sub-10ms language conversion.
- **Phase 7: Dry Run & Custom Inputs:** Trace code execution line-by-line using custom test suites.
- **Phase 8: AI Code & Complexity Review:** Receive 4-part architectural review, time/space complexity $O(N)$, and readiness score.
- **Phase 9: Spaced Repetition Scheduling:** Automated SuperMemo SM-2 scheduling for future memory retention.

3. Alex — Google Senior Staff AI Mentor Architecture

Alex is implemented as a dedicated service layer in `apps/tutor/services/ai.py`. Key sub-modules include:

- **Progressive 3-Level Nudge Engine:** Level 1 (Nudge question) → Level 2 (Pattern reveal) → Level 3 (Pseudocode). Never dumps direct code.
- **Pre-Code Approach Validator:** Warns users if attempting inefficient brute force when optimal pattern exists.
- **Socratic Debugging Engine:** Asks targeted questions on failed test runs rather than giving answers.

- **'Explain It Back' Evaluation:** Evaluates user's pattern understanding (Score $\geq 60\%$ unlocks editorial).
- **Post-Submission Code Audit:** Generates 4-part review (What Went Well, Potential Improvements, FAANG Notes, 4-Metric Scorecard).

4. Topic Mastery Hub & Single Source of Truth

- **Strict AlgoDSA Roadmap Progress:** Visual progress bar and mastery badges (■ Beginner, ■ Improving, ■ Intermediate, ■ Advanced, ■ Mastered) depend **STRICTLY** on AlgoDSA curated roadmap completion. External platforms never inflate the denominator.
- **Separated Platform Metrics:** Each topic header displays *AlgoDSA Roadmap Solved*, *LeetCode Solved*, *GFG Solved*, and *Combined Total Unique Solved*.
- **Multi-Platform Deduplication:** Problems solved on multiple platforms (e.g., Two Sum) merge into a single card with checkmark badges (AlgoDSA ✓ LeetCode ✓ GFG ✓).
- **Pagination & Load More:** Renders 20 cards initially with a clean *Load More (+N remaining)* button to handle 100+ solved questions per topic smoothly.

5. SuperMemo SM-2 Spaced Repetition Engine

AlgoDSA implements the SuperMemo SM-2 algorithm in `apps/progress/services/spaced_repetition.py` to guarantee long-term retention:

$$I(1) = 1, I(2) = 6, I(n) = I(n-1) * EF$$

The Easiness Factor (EF) updates dynamically based on review performance (0-5 rating scale). Items due for revision appear automatically in the **Review Due** tab on the user's workstation.

6. Technical Stack & State Isolation Architecture

- **Backend Engine:** Django 5.x, Python 3.12, SQLite / PostgreSQL, RESTful JSON API.
- **Frontend Architecture:** Vanilla CSS Design System, Alpine.js reactive components, Lucide Icons, Chart.js.
- **Monaco Hero Editor:** Plain JS MonacoManager closure outside Alpine proxy reactivity to prevent browser freezes; isolated storage keys (`algodsa_code_{userId}_{problemId}_{language}`); full unescaping pipeline (`\n, \t, \"`).